Technical guide

BuildTrust AI is a local construction finance sandbox built with React/TypeScript, FastAPI/Python, SQLAlchemy, Alembic and SQLite. It reads real file contents and uses deterministic calculations. It never connects to a bank.

Start with the README, features, architecture and API guide.

Profiles and installation

Profile	Start	Data and login
Public fictional demo	scripts/try_demo.ps1; localhost port 8003	data/public-demo; four public README passwords
Private workspace	scripts/setup.ps1, then scripts/run_demo.ps1; port 8000	data/; generated private passwords
Development	scripts/run_dev.ps1; frontend port 5173	Configured private data; Vite proxies API calls
Isolated verification	Test/evidence wrappers	Separate temporary or uniquely named data folders

Use Python 3.12, Node.js 24 and uv. Setup installs locked dependencies and builds the frontend. Initial downloads are required. Default runtime operation needs no external model or paid API.

The public launcher requires a marked profile and expected accounts. It refuses unrelated data or changed accounts rather than overwriting passwords. Repeat startup keeps records. The public settings use built-in defaults and ignore private environment/.env values, including reserve, expiry, workers and parser bounds. Paid keys are disabled and explanations stay deterministic. Signing secrets remain random and private.

Services and contracts

FastAPI composes focused routers and domain services. Models share one SQLAlchemy Base. Pydantic defines validated requests and authoritative responses. OpenAPI and frontend API types are generated from the app.

POST /agent/analyze saves a workflow and returns its ID. The UI polls saved steps. Analysis can prepare a proposal but stops for human approval. Expensive parsing and password work stay outside ordinary financial write locks. Publication rechecks authority and source versions.

Financial storage

Money is stored as integer minor units and returned as decimal strings. Decimal calculations use ROUND_HALF_UP: money to two decimal places, published quantities/rates to six. Round line amounts before summing. Bound numeric length, digits and exponent before expansion.

Events use UTC timestamps. Business dates use Asia/Dhaka. BDT is the default currency. Estimating/reporting FX needs dated evidence; settlement does not convert currencies.

Users, memberships, suppliers, documents, revisions, procurement, acceptances, invoices, proposals, reservations, balances, payments, journals, workflows and audit retain source links. Estimate versions, observations/selections, measurements, controls and approvals keep their own evidence. Legacy receipt bodies and audit payloads are preserved.

Transactions and upgrades

SQLite enables foreign keys, WAL and bounded lock waits. Financial writes use BEGIN IMMEDIATE. Settlement commits cash, payment, allocations, payable changes, ledger, receipt and success audit together. Failure rolls

back money; denied/failed attempts are audited separately.

The migration head is 0018. The security migration adds sessions, recovery tokens, admission counters and usage reservations. Sign in again after upgrading; earlier tokens are rejected. Startup takes a consistent online backup before upgrading a populated database. Migration 0008 rebuilt payment constraints for installments; later migrations add procurement/correction records and indexes. Original payments, receipts and audit history remain unchanged. Rehearse backup, migration, integrity and recovery on a copy first.

Documents, imports and OCR

Files are limited to 10 MB. The request envelope is bounded before multipart parsing. JSON depth/structure and import row/column counts are bounded. Required/duplicate headers are checked before rows. Failed imports leave drafts unchanged. XLSX formulas and oversized expansion are rejected.

PDF children have page, raster, output, time, 512 MiB memory and 45 CPU-second bounds. PDFs allow at most 50 pages. Normal parsing has a 15-second parent timeout; OCR allows 45 seconds and at most five scanned pages per upload. Resource bounds are not antivirus.

Originals use generated private storage names. PDF previews are authenticated images; text is escaped. Fields retain source/page/row evidence where available and correction history. Procurement needs authorized confirmation. Invoice text cannot update a beneficiary.

OCR is optional. Explicit setup verifies models against ocr-models.lock.json. Runtime does not download models. Missing models preserve manual review. Real Bengali accuracy needs a representative corpus.

Jobs and integrations

Jobs keep attempts, remote task IDs, leases, heartbeats, backoff and current authority checks. The local scheduler resumes eligible work and fences stale publishers. Five attempts are the ceiling. Known paid IDs can resume retrieval; uncertain submissions require manual reconciliation.

The supported model is one local process. Provider keys stay server-side and never appear in public status. Adapters use fixed origins; there is no arbitrary URL-fetch endpoint. See price sources.

Optional Ollama writes explanations only. Bad references, failures or timeouts fall back to deterministic text. It cannot change amounts, roles, approvals or settlement.

Browser and diagnostics

Routes/charts load as needed. Errors retain status, code, fields, request ID and retry details. Session expiry covers previews and downloads too. Draft guards cover navigation, project/version changes and failed saves. Estimates and receipts have separate print scopes.

Logs record request IDs, method/path/status/duration, parser/job timing and material lock waits. They omit passwords, tokens, keys and document bodies. Growing registers use project-scoped paging, filtering and stable sorting.

Configuration and release

Use .env.example for private settings. Never publish private runtime data, uploads, signing secrets or credentials. Back up the database consistently with its private files and signing material.

Hosted Windows/Ubuntu checks passed for the recorded release. These include native Linux shell setup/startup and fresh Windows private/public installation. Docker configuration is included, but its engine/container validation remains pending. See remaining work. Do not alter Docker's managed WSL distribution for Linux testing.

Use isolated test commands. Recordings require a dedicated database/address and reject the working app port. Packaging checks source fingerprints, actual HTTP results, media evidence and private-file exclusion. Fresh archive installation checks setup, payment, estimating and public-demo behavior.

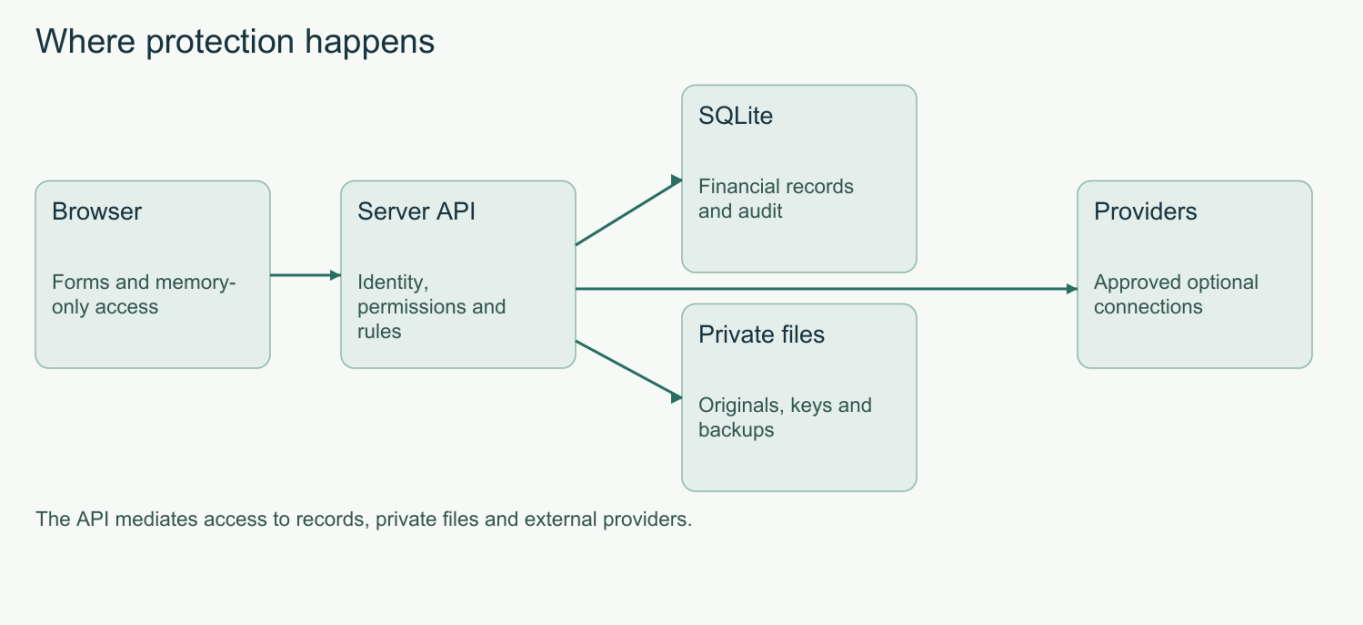
Read current results for actual counts, measurements and external limits. The project uses AGPL-3.0-only with dependency attribution.

See account security for private storage permissions, provider limits and backup/restore commands. Paid-provider credentials alone do not enable requests.

Security architecture extension

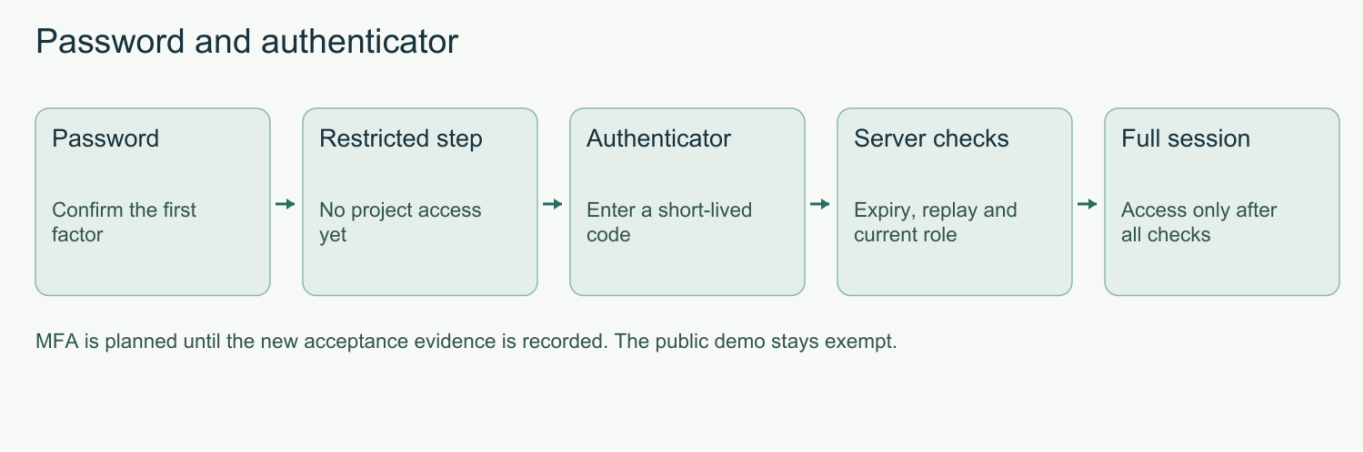
Documentation-first checkpoint: MFA and new request checks are planned. Existing evidence belongs to the recorded baseline; it does not prove unfinished work.

The current application uses memory-only bearer tokens and revocable server sessions. The next migration adds encrypted MFA factors, restricted challenges, hashed recovery codes and session assurance. Planned MFA is not part of the baseline evidence.



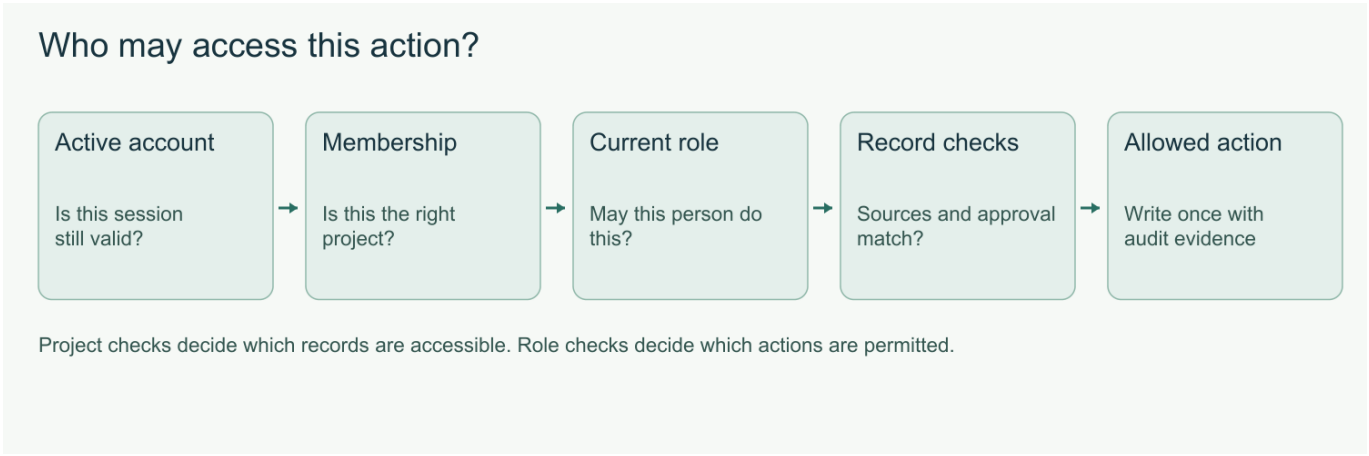
Where protection happens

The API mediates access to records, private files and external providers.



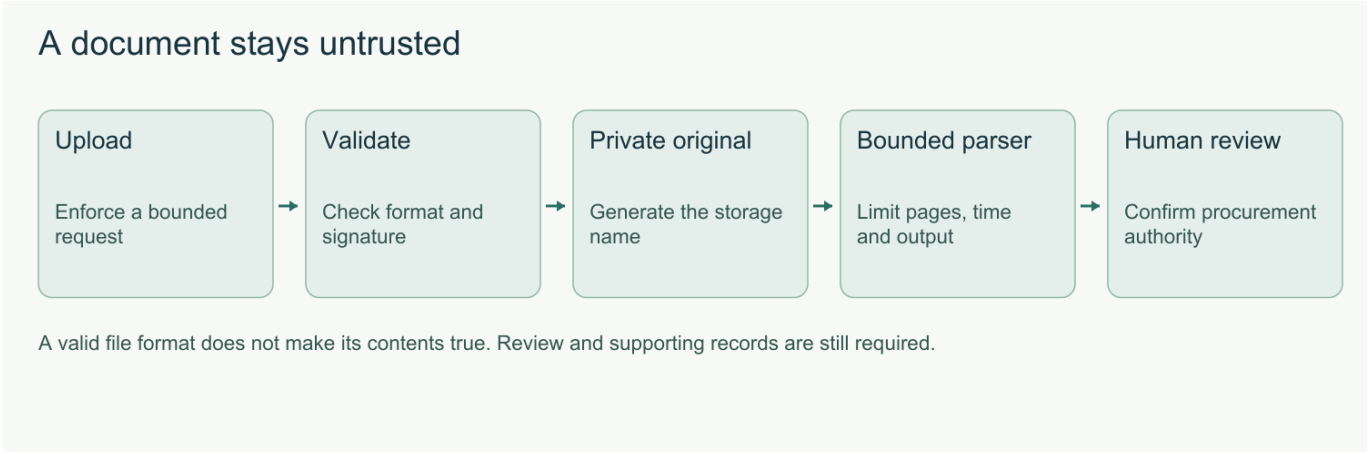
Password and authenticator

Password -> Restricted step -> Authenticator -> Server checks -> Full session.



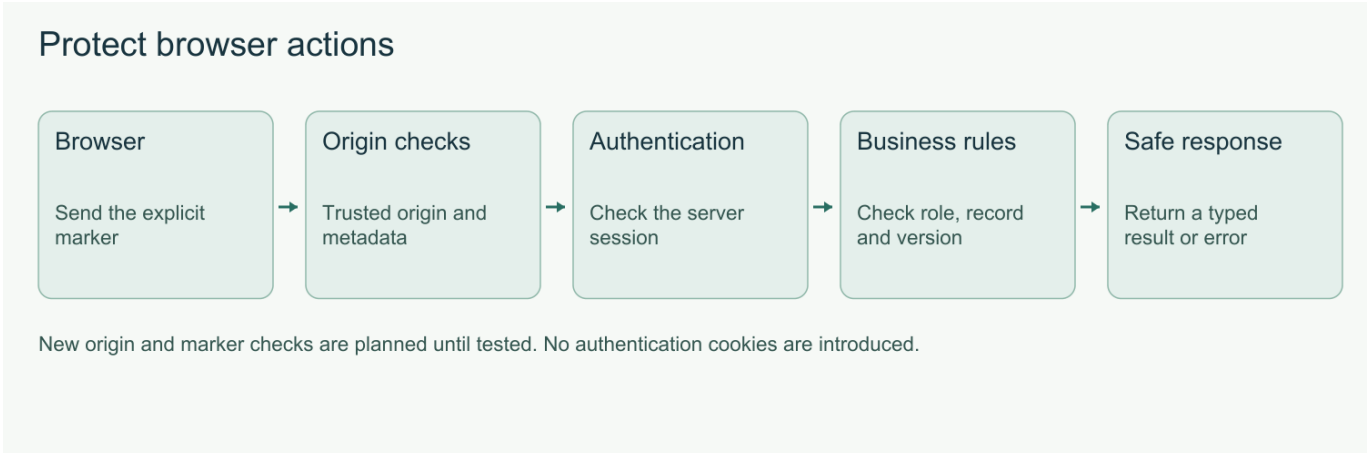
Who may access this action?

Active account -> Membership -> Current role -> Record checks -> Allowed action.



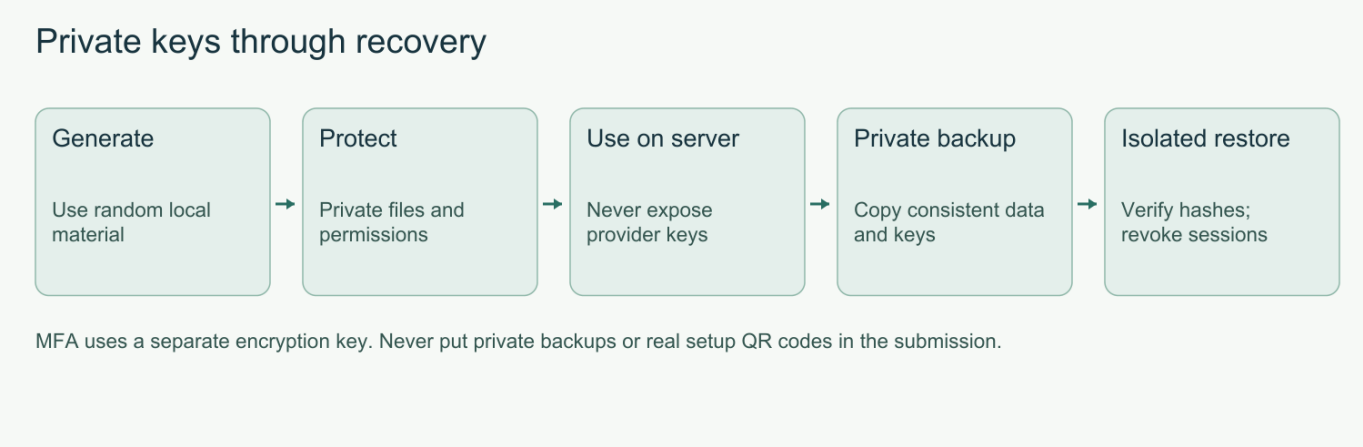
A document stays untrusted

Upload -> Validate -> Private original -> Bounded parser -> Human review.



Protect browser actions

Browser -> Origin checks -> Authentication -> Business rules -> Safe response.



Private keys through recovery

Generate -> Protect -> Use on server -> Private backup -> Isolated restore.

Interfaces and operating rules

- Mutating requests will require X-BuildTrust-Request: 1 and explicit origin checks. No authentication cookies are introduced.

- MFA challenges are restricted to setup or verification; they must never authorize financial actions.

- TOTP uses six digits and 30-second periods. Used periods and recovery codes cannot be replayed.

- A separate private key encrypts MFA factors. Backup and restore must include it; missing keys fail closed.

- Only the local owner may reset MFA. Password recovery alone cannot remove it.

- SQLite keeps project isolation in server queries. Native PostgreSQL RLS is a separate deferred release.

- No incoming webhook receiver is enabled.

- The public-demo credentials in README remain unchanged and exempt from MFA.

See the 19 distinct control entries and recorded verification.